Advanced Topics in Machine Learning - PA5

Ahmad Ashraf [1] **Sarim Malik** [2]

Abstract

Aligning large language models (LLMs) with human values is critical for producing helpful, honest, and safe outputs. Common alignment strategies include Proximal Policy Optimization (PPO), Direct Preference Optimization (DPO), and Group Relative Policy Optimization (GRPO), each with distinct training dynamics, efficiency, and biases. In parallel, mechanistic interpretability (MI) seeks to uncover the internal representations of neural networks, with Sparse Autoencoders (SAEs) enabling the identification of disentangled, *monosemantic* features. This work presents a systematic evaluation of multiple LLM decoding techniques and LLM alignment regimes alongside an analysis of universal internal representations via Universal Sparse Autoencoders (USAEs). Our study aims to (i) identify pros and cons of several LLM decoding strategies like Beam Search, Top-K Sampling, (ii) evaluate the training dynamics and biases of LLM alignment techniques, and (iii) empirically test the Platonic Representation Hypothesis across distinct architectures, providing insights into both aligned model behavior and internal model representations.

Our codebase with complete experiments can be found here: `https://github.com/s-malix21/atml-fall2025` and backup: `https://github.com/ATML-AshrafxSarim/PA5`.

1. Introduction

Large language models (LLMs) have achieved remarkable performance across a wide range of natural language tasks. However, their unaligned outputs can often be unhelpful, unsafe, or misleading. Reinforcement Learning from Human Feedback (RLHF) addresses this challenge by aligning LLMs with human values, typically defined along the axes of *helpfulness*, *honesty*, and *harmlessness*. This alignment is essential for ensuring that LLM outputs not only generate fluent text but also adhere to user expectations and ethical constraints.

Pretrained LLMs, while powerful in next-token prediction, often produce responses that are biased, verbose, or factually incorrect. To mitigate these issues, several alignment strategies have been developed. Proximal Policy Optimization (PPO) leverages reinforcement learning with a learned reward model to fine-tune the LLM policy, balancing exploration with stability via a clipped likelihood objective. Direct Preference Optimization (DPO) simplifies the process by eliminating the reward model entirely and directly optimizing the LLM on human preference pairs, providing computational efficiency and stability. Group Relative Policy Optimization (GRPO) combines reinforcement learning with group-relative advantage estimation, removing the need for a separate value function while maintaining performance on reasoning tasks.

While these alignment methods improve LLM outputs along helpfulness, honesty, and harmlessness axes, the internal mechanisms driving these behaviors remain largely opaque. Understanding how models internally represent knowledge is critical, as it allows us to evaluate whether different alignment strategies induce consistent internal representations across models.

Mechanistic interpretability (MI) is an emerging field that seeks to reverse-engineer neural networks into human-understandable components, revealing the subcircuits underlying specific behaviors. Despite its early state, several hypotheses and contributions have been proposed to explain how networks organize knowledge internally. One prominent idea is the *Platonic Representation Hypothesis*, which suggests that independently trained models converge toward shared internal representations of the world. Sparse Autoencoders (SAEs), and in particular Universal Sparse Autoencoders (USAEs) trained across multiple models, provide a key tool for observing these representations and quantifying the universality of learned features. For example, a ResNet and a Vision Transformer trained on ImageNet may encode concepts such as “wheel” or “fur texture” in comparable ways, despite architectural differences.

In this paper, we perform a systematic evaluation of LLM alignment and internal representations of models through USAEs. Specifically, we investigate:

1. The impact of different decoding strategies on response quality and length compliance across finetuned LLMs.

2. The comparative performance of alignment techniques (PPO, DPO, GRPO) in mitigating verbosity bias, reward hacking, and overall response quality.

3. The ability of sparse autoencoders to identify shared latent features across multiple models, providing evidence for or against the Platonic Representation Hypothesis.

Through this analysis, we provide insights into both the functional and mechanistic behavior of aligned LLMs, bridging output evaluation with internal representation analysis.

2. Methodology

2.1. LLM Decoding Strategy Analysis

2.1.1. Model Architecture and Experimental Setup

We employed the SmolLM2-135M-Instruct as our foundation language model for all decoding experiments. Computations ran on a CUDA-enabled GPU using torch.float16 precision to optimize memory usage. The evaluation dataset consisted of 15 diverse instructional prompts spanning machine learning, science, health, and technology. To assess generation quality automatically, we utilized the **OpenAssistant/reward-model-deberta-v3-large-v2** reward model. All generations were constrained to a maximum sequence length of 80 tokens, with early stopping triggered upon detection of the EOS token.

2.1.2. Decoding Strategies

We implemented and compared four distinct decoding strategies. Greedy Search served as the deterministic baseline, selecting the token with the highest probability at each timestep via an argmax operation. Beam Search maintained a beam width of $B = 4$, utilizing length-normalized cumulative log-probabilities to track the top-B sequences. For stochastic methods, we evaluated Top-K Sampling ($K = 50$) and Top-P (Nucleus) Sampling($P = 0.9$), applying temperature scaling to the logits prior to softmax normalization. To analyze the trade-off between diversity and quality, we performed temperature ablation across the range $T \in \{0.2, 0.5, 0.8, 1.0, 1.2\}$.

2.1.3. Evaluation Metrics

We evaluated performance using both lexical diversity and semantic quality metrics. Distinct-N measured lexical diversity as the ratio of unique N-grams to total N-grams generated, defined for $N \in \{1, 2, 3\}$ as:

$$\text{Distinct-}N = \frac{|\text{unique } N\text{-grams}|}{|\text{total } N\text{-grams}|} \tag{1}$$

Response quality was quantified using the DeBERTa-based reward model, providing scalar scores for coherence and preference alignment. We assessed Across-Prompt Diversity by aggregating single generations from all 15 prompts at fixed $T = 0.8$. Coversely, Within-Prompt Diversity examined 10 independent generations for a single fixed prompt to quantify inter-sample variation. Statistical rigor was ensured through pairwise t-tests and variance analyss.

2.1.4. Supplementary Ablation Studies

We conducted additional studies to test hyperparameter sensitivity and stability. Beam width analysis tested $B \in \{2, 4, 8, 16\}$ to gauge its impact on generation quality. Similarly, we varied sampling parameters with $K \in \{10, 30, 50, 100\}$ and $P \in \{0.7, 0.85, 0.95, 0.99\}$ at a fixed temperature. Finally, we benchmarked computational efficiency by recording tokens-per-second and latency, while stability was tracked via the coefficient of variation in reward scores across methods.

2.2. LLM Alignment

We assess the performance of three different LLM alignment schemes, namely: Direct Preference Optimization (DPO), Proximal Policy Optimization (PPO), and Group Relative Policy Optimization (GRPO).

2.2.1. Baseline Setup

For our experiments, we utilize the `SmolLM2-135M-SFT-Only` model from HuggingFace. This model was selected to provide an efficient balance between computational cost and generation quality for our alignment tasks. The alignment dataset used is Intel's Orca DPO Pairs. Since SmolLM2 is finetuned on instruction-style data, we utilize its tokenizer to consistently format prompts using the provided ChatML chat template, ensuring that the model correctly interprets instruction boundaries and knows when to generate responses.

All experiments were executed using the suite of alignment algorithms available in the `trl` library, maintaining consistent experimental conditions wherever applicable. To further reduce computational overhead, all models were quantized to 8-bit precision (except for PPO), and LoRA adapters were applied with fixed hyperparameters of $r = 16$ and $\alpha = 32$, targeting all linear projection layers.

2.2.2. Alignment Schemes and Setup

Direct Preference Optimization (DPO). DPO does not rely on an explicit reward model, unlike the later alignment schemes. Instead, it directly minimizes the following loss function:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)$$

where y_w and y_l denote the preferred (winner) and less preferred (loser) responses, π_θ is the current policy, π_{ref} is the reference model, and β controls the strength of preference alignment. The β parameter is the most critical hyperparameter in DPO, and we use a value of 0.5 to provide a balance between enforcing preference alignment while not deviating excessively from the reference model.

We load the initial SFT-Only model as the reference model and train the aligned model for 2 epochs on a subset of the dataset (2,500 preference pairs) to reduce training time. A maximum sequence length of 512 tokens is set to prevent runaway generation. We use a learning rate of 5×10^{-5}, along with training specific parameters such as batch size of 2 and gradient accumulation steps of 4, in order for smooth optimization.

Proximal Policy Optimization (PPO). PPO requires a reward model to provide scalar feedback for each generated completion. During training, the policy model produces candidate responses which are then evaluated by the reward model to obtain a scalar reward. The value model critiques these rewards and guides the policy update through the standard PPO objective:

$$\mathcal{L}_{\text{PPO}}(\theta) = -\mathbb{E}_t[\min(r_t(\theta)A_t,\, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\, A_t)]\,,$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio, and A_t is the advantage estimated by the value model.

We first train a reward model on a subset of 5,000 preference pairs for a single epoch, assigning a reward of 1 to the preferred completion and 0 to the rejected one. PPO alignment is then applied using the original SFT-Only model as both the policy and reference model. We investigate two reward mechanisms: (1) *Sparse rewards*, where reward is given only at the end-of-sequence token, and (2) *Dense rewards*, where reward is distributed across all valid tokens with an additional KL penalty to encourage proximity to the base model. For policy optimization, we adopt a learning rate of 1×10^{-4}, a batch size of 2 with gradient accumulation steps of 4, resulting in an effective batch size of 8. We align the model for a single epoch over 2,500 training samples with 500 held out for evaluation. To prevent the policy from drifting too far from the reference behavior, the original SFT-only model is also used as a reference policy for computing an additional KL penalty within the PPO loss.

Group Relative Policy Optimization (GRPO). GRPO removes the need for a learned critic, unlike PPO, by leveraging the reward model directly in a group-wise comparison. For each prompt, the model generates k candidate completions, all of which are scored by the reward model. The objective encourages the policy to increase the likelihood of higher-reward responses relative to the group distribution. The loss can be expressed as:

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\mathbb{E}_x\, \mathbb{E}_{y_i \sim \pi_\theta(\cdot|x)} \left[\left(R(y_i) - \overline{R}\right) \log \pi_\theta(y_i \mid x)\right],$$

where $R(y_i)$ is the scalar reward assigned by the reward model and $\overline{R}$ is the mean reward across the k sampled completions for the same prompt.

We reuse the same reward model trained for PPO. For each input prompt, we sample $k = 4$ completions, enabling relative scoring within the group. Due to the substantially higher computational demands of GRPO, we limit training to 500 prompts, sufficient for the model to learn basic preference alignment behaviors. The model is trained with a learning rate of 5×10^{-5}, a batch size of 2, and gradient accumulation steps of 4 to maintain stable optimization as also noticed in previous experimental setups.

2.2.3. Evaluation Strategy

We evaluate all aligned models using a set of 50 test prompts designed to resemble the preference dataset. These prompts cover a wide range of types, including factual, open-ended, brief, explanatory, and intentionally adversarial "hack" prompts. The complete set of prompts is provided in the Appendix. Responses are generated from all models on this test set, and evaluation is conducted across several dimensions. For all tests, generation is limited to a maximum of 256 tokens.

Catastrophic Forgetting. To assess catastrophic forgetting, we compute two key metrics: (1) the KL divergence between the aligned policy model and the frozen reference model, and (2) perplexity on the original dataset.

The KL divergence for a given prompt x is defined as:

$$D_{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{ref}}) = \sum_y \pi_\theta(y \mid x) \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}.$$

Perplexity is calculated as:

$$\text{Perplexity} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N} \log \pi_\theta(y_i \mid x_i)\right),$$

where N is the total number of tokens in the evaluation set. For both metrics, lower values indicate that the model has retained its prior capabilities and has not deviated significantly from the reference model.

Verbosity Bias. Verbosity bias is assessed by analyzing the distribution of response lengths. We compute the mean, median, and standard deviation of token counts as well as their skewness and kurtosis values across all responses. We also assess the performance of each model by stratifying with respect to each query type. Additionally, we perform a length compliance test on 10 prompts with explicit word limits. A margin of ± 20 words is allowed, and model performance is assessed based on adherence to the specified limits.

Reward Hacking. To test for reward hacking, we first evaluate whether the PPO reward model is overparameterized. For this, we create perturbed versions of normal prompt–response pairs that preserve semantic content but vary surface form (e.g., adding filler phrases, reordering sentences, or injecting common alignment keywords). If these superficial changes significantly affect the reward, the model is considered overfitting to shallow cues.

Next, we assess vulnerability using targeted hack prompts from the test set. These prompts are designed to trick the model in providing gibberish responses that might be rewarded by the reward model. The rewards assigned by the PPO reward model are examined, and lower reward scores are considered better, indicating reduced susceptibility to reward hacking.

2.3. Sparse Autoencoders

2.3.1. Experimental Setup and Architecture

We utilized two distinct pre-trained architectures to analyze representation alignment: a Convolutional Neural Network (ResNet-18) and a Vision Transformer (ViT-B-16), both initialized with ImageNet weights from torchvision. Experiments were conducted on the CIFAR-10 dataset, applying standard ImageNet preprocessing including resizing to 224×224 and normalization. To balance computational efficiency with experimental validity, we subsampled the dataset to 5,000 training images and 1,000 test images. We extracted internal activations via forward hooks registered at the final convolutional block of ResNet-18 (layer4, 512 dimensions) and the final encoder block of ViT-B-16 (encoder_layer_11, CLS token, 768 dimensions). All computations were performed on a CUDA enabled GPU using mixed-precision torch.float16 optimization.

2.3.2. Universal Sparse Autoencoder (USAE)

We implemented a Universal Sparse Autoencoder designed to map diverse model embeddings into a shared dictionary space of size $m = 2048$. The architecture consists of $M = 2$ encoders $\Psi^{(i)} : \mathbb{R}^{d_i} \rightarrow \mathbb{R}^m$ and $M = 2$ decoders $D^{(j)} : \mathbb{R}^m \rightarrow \mathbb{R}^{d_j}$, each equipped with learnable bias terms $b_{pre}^{(i)}$ and $b_{dec}^{(j)}$. Sparsity is enforced using a Top-K activation function ($k = 32$), generating shared latent codes Z as follows:

$$Z = \text{TopK}\left(W_{enc}^{(i)}(A^{(i)} - b_{pre}^{(i)})\right) \tag{2}$$

Training followed a randomized protocol where a source model is selected stochastically per batch, and the resulting code Z is decoded to reconstruct activations for all target models. We minimized the aggregate Mean Squared Error (MSE) loss:

$$\mathcal{L} = \sum_{j=1}^{M} ||A^{(j)} - \hat{A}^{(j)}||_F^2 \tag{3}$$

Optimization utilized the Adam optimizer with a learning rate of 1×10^{-3}, cosine annealing scheduling, and a batch size of 128 over 10 epochs.

2.3.3. Metrics for Universality and Alignment

To quantify cross-model alignment, we computed the R^2 reconstruction matrix, where $R_{i,j}^2$ measures the variance explained when reconstructing Model j's activations using codes derived from Model i. Feature universality was assessed using the Normalized Firing Entropy (FE_k) for each feature k:

$$FE_k = -\frac{1}{\log(M)} \sum_{i=1}^{M} p_k^{(i)} \log(p_k^{(i)}) \tag{4}$$

Here, $p_k^{(i)}$ denotes the proportion of total activations for feature k attributable to Model i. Values near 1.0 indicate universal features shared across architectures, while values near 0.0 imply model-specificity. We further analyzed alignment through co-firing proportions, measuring the frequency with which features activate simultaneously across models for identical inputs. The "Alignment Tax" was calculated by comparing USAE reconstruction performance against independent SAE baselines to quantify the fidelity trade-off incurred by enforcing a shared latent space.

2.3.4. Supplementary Analysis

We validated semantic coherence via nearest-neighbor retrieval in the shared Z-space, assessing class consistency across CIFAR-10 categories. A class-stratified analysis of cross-model reconstruction (ResNet $\rightarrow$ ViT) was performed to detect category-dependent universality. To test functional importance, we conducted a "lobotomy" experiment, selectively ablating high-entropy versus low-entropy features and measuring the impact on cross-model translation. Finally, feature splitting was monitored using Jaccard similarity between activation patterns, and computational efficiency was benchmarked via convergence rates and inference latency.

3. Results

3.1. LLM Decoding Strategy Analysis

3.1.1. Decoding Strategy Comparison at Fixed Temperature

We compared the four decoding strategies at a fixed temperature of $T = 0.8$. Table 1 presents the performance metrics. Beam Search ($B = 4$) achieved the highest quality, with a mean reward score of 4.11. However, this quality came at a significant computational cost; Beam Search was the slowest method, generating only 5.3 tokens per second. In contrast, Greedy Search, Top-K, and Top-P all achieved speeds exceeding 21 tokens per second.

Regarding diversity, the deterministic methods (Greedy and Beam) produced identical outputs for repeated runs of the same prompt, resulting in a unique response rate of 1.0. The sampling methods (Top-K and Top-P) generated distinct text in every trial (10.0 unique responses). Top-K ($K = 50$) showed the highest within-prompt entropy (4.84).

Stability analysis (Table 2) indicates that Greedy Search is the most stable method with the lowest coefficient of variation (CV = 0.35). Top-K was the least stable (CV = 3.27) and showed a very wide range in reward scores. We also observed a strong positive correlation ($r = 0.99$) between output length and reward for Top-K, suggesting its high scores may be biased by simply generating longer text.

3.1.2. Temperature Ablation and Diversity-Quality Tradeoffs

Table 10 details the impact of temperature on generation. Lower temperatures favored quality. Both Top-K and Top-P achieved their highest rewards at $T = 0.2$ (2.39 and 2.58, respectively). As the temperature increased to $T = 1.2$, quality degraded significantly, resulting in negative reward scores for both methods.

Higher temperatures improved diversity metrics but often at the cost of coherence. For Top-K, increasing T to 1.0 raised the Distinct-2 score to 1.00, but the average length dropped to 26.0 tokens, indicating that the model began generating short, nonsensical sequences (collapse). Top-P maintained better length stability across temperatures, though its reward score still dropped as T increased. The data suggests that for this model, a temperature range between 0.2 and 0.8 offers the best balance between quality and diversity.

3.1.3. Hyperparameter Sensitivity and Computational Analysis

We examined the sensitivity of each method to its core hyperparameters in Table 11. For Beam Search, increasing the beam width (B) from 2 to 8 improved the reward score from 1.90 to 2.48. However, increasing the width further to 16 caused a drop in reward (2.08) and output length (76.0), suggesting a point of diminishing returns.

For Top-K sampling, a value of $K = 50$ yielded the highest reward (3.96), outperforming both lower ($K = 10$) and higher ($K = 100$) settings. Top-P performed best with a stricter threshold; $P = 0.70$ resulted in a high reward of 6.63, significantly better than the standard $P = 0.95$ setting (2.58). This indicates that filtering the tail of the distribution more aggressively helps this specific model maintain coherence.

3.1.4. Method-Specific Behavior and Trade-off Analysis

The results highlight a clear trade-off between determinism and diversity. Deterministic methods (Greedy and Beam) provide high quality and stability but fail to generate variations for the same prompt. Beam Search offers the highest quality overall but is computationally expensive, making it less suitable for latency-sensitive applications.

Stochastic methods (Top-K and Top-P) solve the diversity issue but introduce variance in quality. Top-K showed high instability and a susceptibility to length bias. Top-P appears more robust, especially at lower P thresholds (0.70), offering a practical middle ground that maintains generation speed while providing diverse outputs.

3.2. LLM Alignment

3.2.1. Catastrophic Forgetting

Table 3. Catastrophic Forgetting Evaluation: KL Divergence and Perplexity

Model	KL Divergence	Perplexity
PPO (Sparse)	0.0015	2.0045
PPO (Dense)	0.0012	2.0053
DPO	0.0163	2.1375
GRPO	0.0006	2.0008

Table 3 highlight the KL divergence and perplexity between each aligned policy model and the frozen reference model. Lower values in both metrics indicate stronger retention of the original model's behavior. GRPO achieves the lowest KL divergence and perplexity, suggesting minimal deviation from the base model. Both PPO variants (Sparse and Dense) also maintain close proximity to the reference model, exhibiting only marginal differences. In contrast, DPO shows noticeably higher KL divergence and perplexity, indicating a stronger shift away from the pretrained distribution. These results likely reflect our constrained training setup: shorter training schedules and limited data can restrict the benefits alignment methods such as GRPO, which typically require

Table 1. Comparative performance of decoding strategies. We report Reward scores (Quality), generation speed (Tokens/sec), and diversity metrics (Distinct-2 and Entropy) across both aggregated prompts and within single prompts. **Bold** indicates best performance.

	Efficiency & Quality		Across-Prompt Diversity			Within-Prompt Diversity		
Method	**Tok/s**	**Reward**	**Dist-2**	**Dist-3**	**Entropy**	**Dist-2**	**Unique**	**Entropy**
Greedy	21.2	3.52	**0.93**	**0.98**	4.24	0.10	1.0	3.80
Beam ($B = 4$)	5.3	**4.11**	0.87	0.95	**5.27**	0.10	1.0	3.81
Top-K ($K = 50$)	**21.4**	1.03	0.90	0.96	5.04	**0.74**	**10.0**	**4.84**
Top-P ($P = 0.9$)	21.2	2.53	0.88	0.95	4.77	0.53	**10.0**	4.51

Table 2. Stability and Bias Analysis showing reward distribution stats and Length-Reward correlation (r).

	Reward Score Stability				Bias
Method	**Mean**	**Std**	**Rng**	**CV**	**Corr** (r)
Greedy	3.52	1.22	1.73	**0.35**	–
Beam	**4.11**	2.28	5.96	0.56	-0.07
Top-K	1.03	3.36	6.36	3.27	+0.99
Top-P	2.53	1.90	4.41	0.75	-0.84

more data and training epochs to converge stably without over-drifting from the base model.

3.2.2. Verbosity Bias

Overall Verbosity Statistics. The overall verbosity statistics (Table 4) reveal clear differences in how alignment strategies impact response length. The Base and GRPO models remain strongly top-capped at 256 tokens, as shown by their identical medians and large negative skewness. After that come the PPO models (Sparse and Dense), which reduce this bias slightly, with less skewness and lower mean token counts. However, the median token counts are consistently at or near 256 for most models, except for DPO which has a slightly lower median of 250.5 tokens.

DPO exhibits the lowest mean response length and the least negative skewness, indicating that it diverges most from the max-length mode and is more willing to produce shorter responses when appropriate. However, its higher kurtosis and standard deviation imply less stability. Although DPO shortens many answers, it occasionally produces disproportionately long ones, reflecting inconsistency rather than principled adaptation. In contrast, PPO-Sparse achieves the most balanced shape (lowest kurtosis near zero), suggesting a distribution closer to normality and thus more consistent length control without heavy-tail responses.

Query-wise Response Statistics. Table 5 further breaks down response lengths by query type and demonstrates how alignment strategies differentially affect verbosity. For explanation and open-ended prompts, where longer outputs are acceptable, all aligned models continue to generate till the maximum token budget. Our smaller training regime might indicates that none of the methods successfully learned task-adaptive scaling for high-complexity prompts; instead, they retain a strong bias toward maximal elaboration when the instruction allows it.

In contrast, for factual questions, every aligned model produces substantially shorter responses than the Base model. PPO-Dense, PPO-Sparse, and GRPO roughly reduce verbosity by 130–160 tokens on average, while DPO enforces the strongest compression, reducing the mean to just 79 tokens.

Length Compliance. Results in Table 6 show that all aligned models, except for DPO, perform poorly on length compliance when evaluated with a strict deviation of ±10 words. DPO achieves the highest compliance rate and the lowest mean word count deviation, indicating that it is the most successful at respecting explicit length constraints.

However, it is notable that GRPO reduces the average word deviation more than the other models, and both PPO Dense and PPO Sparse also show moderate improvements in average deviation. This suggests that RL-based alignment methods are somewhat effective in adapting the model output length, even if strict compliance is not achieved. The limited overall performance may be due to the preference dataset used for training, which contains few examples with strict length constraints, preventing the models from learning more precise length control.

Table 4. Verbosity Statistics: Overall Token Counts by Model

Model	Mean	Median	Std	Skew.	Kurt.
Base	217.58	256.0	77.51	-1.69	1.21
PPO (Sparse)	207.92	256.0	87.23	-1.38	0.20
PPO (Dense)	204.38	256.0	88.36	-1.29	-0.05
DPO	179.17	250.5	97.02	-0.74	-1.15
GRPO	211.67	256.0	82.77	-1.42	0.23

Table 5. Verbosity Statistics by Model and Prompt Category (Token Counts)

Category	Base	DPO	GRPO	PPO (Dense)	PPO (Sparse)
Explanation	256.00	245.60	256.00	256.00	256.00
Factual	190.90	79.70	161.20	132.20	127.40
Hack	191.11	167.56	191.39	188.17	199.22
Open-ended	253.50	233.10	254.30	254.10	256.00
Overall	217.58	179.17	211.67	204.38	207.92

Table 6. Length Compliance Evaluation: Compliance Rate and Mean Word Count Deviation

Model	Compliance Rate	Mean Word Count Deviation
Base	0.25	25.25
DPO	0.75	5.75
GRPO	0.25	20.75
PPO (Dense)	0.25	20.75
PPO (Sparse)	0.25	23.75

Table 8. Reward Model Sensitivity to Perturbations (Mean Reward Deltas)

Perturbation Type	Mean Reward Delta
Filler Phrases (Bias)	-0.1352
Alignment Keywords (Bias)	-0.1268
Non-Factual (Robustness Check)	-0.1958
Added Length (Verbosity Check)	-0.0570

3.2.3. Reward Hacking Evaluation

Mean Reward for Hack Prompts. Table 7 reports the average reward assigned by the trained reward model to outputs generated for adversarial "hack" prompts designed to bait alignment-keyword exploitation or overly-safe generic responses. The reward model does not consider these responses desirable. All models achieve negative mean reward, confirming that the reward model is trained correctly.

DPO receives the largest negative reward (-1.3406), which aligns with expectations since it does not incorporate a direct reward model during its alignment procedure, making it less sensitive to reward-optimized behaviors. In comparison, PPO (Dense) and PPO (Sparse) achieve smaller negative rewards (-1.1624 and -1.1975, respectively), reflecting that RL-based alignment allows these models to partially optimize their outputs according to the reward model, reducing susceptibility to naive prompt manipulations. GRPO lies in between (-1.2287), benefitting from repeated scoring using the reward model but still maintaining conservative behavior.

Table 8 presents the mean reward deltas for different types of perturbations applied to factual responses. The results indicate that the reward model is not overly sensitive to superficial cues such as filler phrases or alignment keywords. Non-factual responses are appropriately penalized, and added verbosity has a smaller effect, confirming that the reward model behaves robustly and aligns well with the intended training objectives.

Per-token Reward Analysis. Beyond mean reward alone, we also assess reward efficiency using a reward-per-token metric, which highlights whether models gain reward by being longer or by changing content. Across all models, the values remain negative (Table 9), indicating that longer responses tend to be penalized by the reward model rather than exploited.

Table 7. Reward Hacking Evaluation: Mean Rewards on Hack Prompts

Model	Mean Reward
PPO (Sparse)	-1.1975
PPO (Dense)	-1.1624
DPO	-1.3406
GRPO	-1.2287

Table 9. Reward-per-Token Evaluation Across Models

Model	Reward-per-Token
GRPO	-0.0047
PPO (Dense)	-0.0049
Base	-0.0046
PPO (Sparse)	-0.0049
DPO	-0.0060

3.3. Sparse Autoencoders

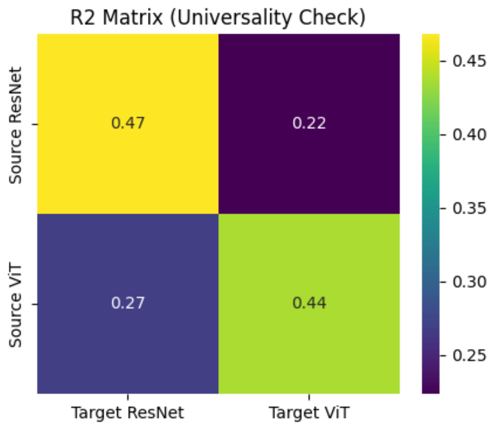

Figure 1. **Cross-Model Reconstruction (R^2) Matrix.** Diagonal elements show self-reconstruction fidelity, while off-diagonals quantify cross-model translation capabilities. The relatively high scores confirm effective universality.

3.3.1. USAE Training and Reconstruction Quality

We first evaluated whether a single shared bottleneck could successfully reconstruct features from two distinct architectures. Figure 1 presents the R^2 reconstruction matrix. The diagonal elements represent self-reconstruction, where the model encodes and decodes back to the same architecture. ResNet achieved a self-reconstruction score of 0.47, while ViT achieved 0.44. These scores indicate that the USAE retains the majority of the functional information required by each model individually.

The off-diagonal elements in Figure 1 quantify cross-model translation. We observed positive R^2 values for both ResNet→ViT (0.22) and ViT→ResNet (0.27). While these scores are lower than self-reconstruction, they are non-zero and significant. This confirms that the shared bottleneck Z successfully captures a universal geometry accessible to both architectures, enabling direct feature translation without paired training data.

3.3.2. Feature Universality Distribution

We analyzed how the 2048 learned dictionary atoms are distributed between the two models. Figure 5 (Left) displays the histogram of Normalized Firing Entropy (FE_k). The distribution is strictly bimodal. We see a large concentration of features near zero entropy, representing model-specific units that fire almost exclusively for one architecture. Conversely, there is a distinct, flat tail of high-entropy features (near 1.0) representing universal units active in both models.

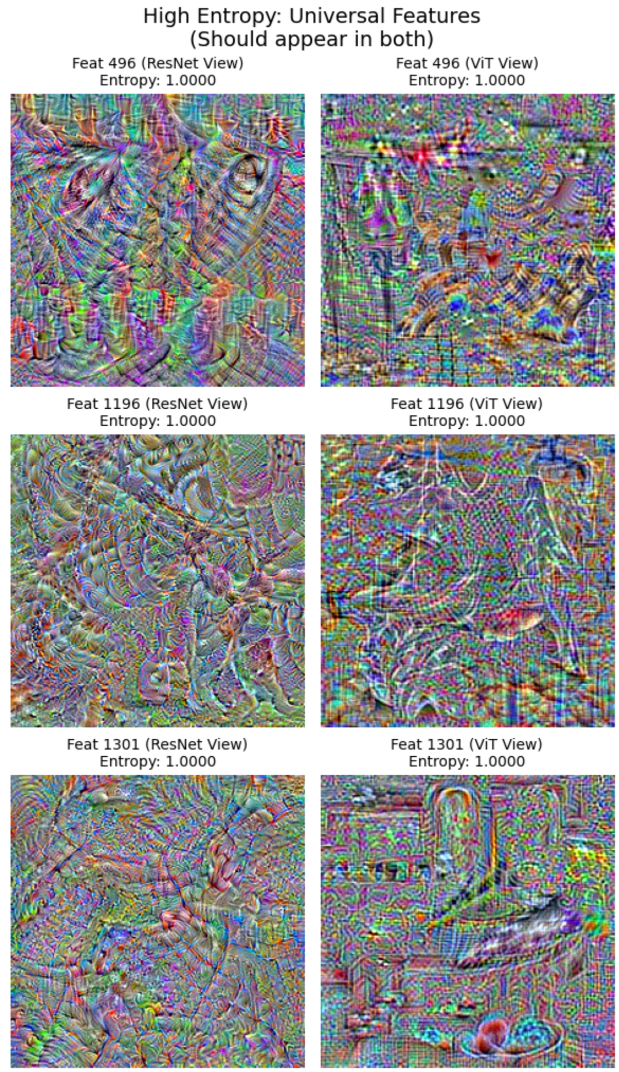

Figure 2. **Universal Feature Visualization (High Entropy).** These features ($FE \approx 1.0$) act as a "bridge" between models. Coordinated Activation Maximization shows that they represent coherent visual patterns active in both ResNet and ViT.

Figure 5 (Right) further illustrates this separation through co-firing proportions. Most features rarely co-fire, confirming that the USAE disentangles the representation into two sets: a local pattren specific to ResNet or ViT, and a shared universal language. This sharp separation challenges the idea of a continuous spectrum of universality, suggesting instead that features tend to split cleanly into specific or shared roles.

3.3.3. Coordinated Activation Maximization

To verify the semantic meaning of these features, we visualized them using Coordinated Activation Maximization.

Figure 2 shows the high-entropy (universal) features. In these cases, the visualizations generated by the ResNet decoder and the ViT decoder are semantically consistent. For instance, Feature 1196 produces similar texture patterns in both views, confirming that these units act as a semantic bridge.

In contrast, Figure 6 visualizes low-entropy (model-specific) features. Here, the owner model produces a clear, high-contrast pattern, while the non-owner model generates a relatively different sort of structure. This visual mismatch explains why the entropy scores in the previous section were near zero; the feature represents a concept that simply does not exist or is encoded differently in the other architecture.

3.3.4. Alignment Tax and Comparative Evaluation

We quantified the cost of enforcing this universality. As shown in the top section of Table 12, the USAE suffers a performance penalty compared to training two independent SAEs. The average reconstruction R^2 dropped from 0.5820 (Independent) to 0.4678 (USAE), representing an "Alignment Tax" of roughly 11.4%. This is the trade-off required to force two diverse geometries into a single latent space.

To determine which features drive cross-model translation, we performed a "Universal Lobotomy" (Table 13). Ablating the high-entropy features resulted in a massive performance drop (0.3675), pushing the R^2 into negative territory. Ablating the low-entropy features caused only a minor drop (0.0712). This confirms that the small subset of high-entropy features carries the bulk of the translational load.

Finally, we analyzed alignment across different semantic categories. Figure 4 shows that rigid classes like "Truck" (0.55) and "Car" align much better than organic classes like "Bird" (0.26). This suggests that mechanical features are more universally represented across CNNs and Transformers than organic textures. Despite these differences, the retrieval results in Figure 3 show that the shared space maintains high-level semantic coherence, successfully clustering concepts like trucks and birds regardless of the source model.

4. Discussion

4.1. LLM Decoding Strategy Analysis

4.1.1. Deterministic Trade-offs

Beam Search ($B = 4$) achieved the highest mean reward (4.11), significantly outperforming Greedy Search (3.52) by exploring multiple potential sequences to avoid local optima. However, this quality came at a severe computational cost: generation speed dropped to 5.3 tokens/sec compared to 21.2 tokens/sec for Greedy, confirming the linear bottleneck inherent in beam decoding. Despite the high reward scores, Beam Search exhibited the lowest lexical diversity (Dist-2: 0.87). This finding aligns with (Holtzman et al., 2020), who argue that maximization-based strategies prioritize high-probability, generic tokens, often leading to repetitive loops rather than genuine fluency. The reward model likely conflated structural grammatical safety with quality, thus it hid the lack of creativity that characterizes deterministic outputs.

4.1.2. Sampling Stability

Top-K ($K = 50$) demonstrated extreme instability (CV: 3.27) and a near-perfect correlation between output length and reward. This indicates the method "gamed" the evaluation by generating long, wandering sequences, effectively validating (Singhal et al., 2023)'s critique of length bias in reward modeling. Furthermore, the variance supports (Holtzman et al., 2020)'s observation that fixed truncation (K) fails to adapt when the probability distribution flattens, forcing the model to sample from an unreliable tail. In contrast, aggressive Nucleus Sampling drastically outperformed standard settings (Reward: 6.63 vs 2.58 for $P = 0.95$). This suggests that for smaller models like SmolLM2, the probability tail contains significant noise; dynamic truncation (P) successfully filters this noise, offering a superior balance of coherence and diversity that is more robust against reward hacking.

4.2. LLM Alignment

4.2.1. Training Dynamics

Overall, DPO remains the fastest and simplest method for aligning an LLM since it avoids reward-model training and RL loops. This ease of deployment makes DPO attractive under tight compute or data constraints. Furthermore, it also showed consistent performance in terms of length compliance and overall results. However, in our experiments, the quality of DPO outputs suffered from a high variance, indicating an inconsistency in the generation behavior when the requests vary in style or requirement.

By contrast, RL-based schemes such as PPO (and by extension GRPO) come with substantial overhead. PPO requires maintaining multiple models (policy, reward, value/critic, reference) simultaneously, which increases memory usage, implementation complexity, and compute cost. Moreover, stable alignment via PPO typically demands extensive sampling (relative to our compute) and is very sensitive to hyperparameters. In our case, we were constrained by a limited preference dataset and a small number of training epochs, which likely prevented PPO (and GRPO) from reaching their full potential. This was also why these models exhibited lower KL divergence values and perplexity scores as well.

4.2.2. Verbosity Bias and Reward Hacking

Our results indicate that almost all models reduce verbosity bias compared to the base model, but none completely eliminate it. Among the aligned models, DPO performs the best, primarily because it does not rely on a reward model. Unlike sparse PPO, DPO does not need to pad responses to maximize reward; its training objective is relative by comparing preferred versus rejected responses rather than absolute.

In contrast, PPO and GRPO exhibit higher verbosity biases due to their training regimes. Reward models can implicitly encourage longer responses because accumulating more tokens can increase the total reward for a generation. Additionally, our preference dataset often contains long responses, which may lead the reward model to associate higher length with better outcomes. Even if the model avoids unnecessary lengthening of factual statements (it negatively rewards lengthened statements), it may have implicitly learned to favor longer responses in some contexts.

Our experiments indicate that the reward model functions correctly; however, its interaction with the nature of the dataset and the training regimes, especially under computational constraints in PPO and GRPO, contributes to the observed verbosity bias of these aligned models.

4.3. Sparse Autoencoders

4.3.1. The Alignment Tax and the Platonic Ideal

Our experiments reveal a tangible cost to universality. The Alignment Tax was an 11.4% drop in reconstruction fidelity compared to independent SAEs, which indicates that the ”Platonic Representation Hypothesis” (Huh et al., 2024) is an idealization rather than a perfect reality for models of this scale. While distinct architectures like ResNet and ViT do converge on shared high-level concepts (evident in the successful retrieval of semantic neighbors), their internal geometries remain distinct enough that forcing them into a shared bottleneck degrades precision. This friction suggests that while models learn the same 'what' (concepts), they fundamentally disagree on the 'how' (implementation). The shared dictionary acts less like a perfect map, prioritizing semantic consensus over the fine-grained, texture-level details that Convolutional inductive biases preserve but Transformers might discard (Raghu et al., 2021).

4.3.2. Bimodality and the Nature of Shared Features

The firing entropy distribution was strictly bimodal, meaning features were almost always fully shared or fully specific to one model. This suggests that universality is not a spectrum. A feature is either a clear visual concept usable by both architectures (Olah et al., 2020), or it is specific to how one particular model works. When we visualized the specific features, they usually looked like random noise to the other model. This implies they are likely just artifacts or ”polysemantic” noise (Bricken et al., 2023) that the USAE correctly filters out. By separating these, the model avoids forcing a match between features that simply do not align.

4.3.3. Is Universality Emergent or Fundamental?

We also examined if universality requires massive models or if it happens naturally in smaller ones. Since we successfully aligned small models (ResNet and ViT) on CIFAR-10, it appears universality is a fundamental property of learning. Even without being ”foundation models,” they learned shared features for objects like cars and trucks. This aligns with (Gurnee et al., 2024), who argue that the data itself drives shared representations. However, the models struggled to align on organic classes like birds. This suggests that while universality exists at small scales, larger models are probably needed to overcome specific biases (like CNNs relying on texture (Raghu et al., 2021)) to reach a fully shared Platonic reprsentation (Huh et al., 2024).

5. Conclusion

This study provides a systematic evaluation of model behavior across three complementary dimensions: generation strategies, alignment techniques, and internal representations. Our analysis reveals fundamental trade-offs inherent in each domain. Decoding strategies exhibit a clear tension between computational efficiency and output quality, with sampling methods offering superior diversity at the cost of stability. Alignment techniques demonstrate varying effectiveness under resource constraints, where DPO emerges as the most practical approach for length compliance and verbosity control, while RL-based methods require substantial computational investment to reach their potential. Most significantly, our sparse autoencoder experiments provide empirical evidence for partial universality in neural representations. The observed bimodal distribution of feature entropy and the measurable alignment tax suggest that while distinct architectures do converge on shared high-level concepts, this universality comes with quantifiable costs and remains incomplete at smaller scales. These findings collectively underscore the importance of understanding both the functional outputs and internal mechanisms of neural networks, particularly as the field moves toward deploying increasingly complex models in practical applications where interpretability and reliability are significant.

References

Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Li, T., et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.

Gurnee, W., Nanda, N., Pauly, M., Harvey, K., Trott, D., and Bertsimas, D. Universal neurons in gpt2 language models. *arXiv preprint arXiv:2401.12181*, 2024.

Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*, 2020.

Huh, M., Cheung, B., Wang, T., and Isola, P. The platonic representation hypothesis. *arXiv preprint arXiv:2405.07987*, 2024.

Olah, C., Cammarata, N., Schubert, L., Goh, G., Petrov, M., and Carter, S. Zoom in: An introduction to circuits. *Distill*, 5(3):e00024–001, 2020.

Raghu, M., Unterthiner, T., Kornblith, S., Zhang, C., and Dosovitskiy, A. Do vision transformers see like convolutional neural networks? In *Advances in Neural Information Processing Systems*, volume 34, pp. 12116–12128, 2021.

Singhal, P., Goyal, T., Xu, J., and Durrett, G. A long way to go: Investigating length correlations in rlhf. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

A. Appendix

A.1. Supplementary Material - LLM Decoding Strategy Analysis

Table 10. Impact of temperature (T) on generation metrics. High T increases diversity but degrades reward.

Method	T	Dist-1	Dist-2	Rew.	Ent.	Rep.	Len.
Top-K	0.2	0.64	0.93	**2.39**	3.64	0.35	69.0
	0.5	0.77	0.95	1.58	3.28	0.21	49.3
	0.8	0.73	0.95	0.71	**3.80**	0.25	68.0
	1.0	0.92	**1.00**	-2.91	3.15	0.08	26.0
	1.2	**0.93**	0.99	-2.02	3.09	**0.07**	34.8
Top-P	0.2	0.73	0.96	2.58	**3.80**	0.27	70.0
	0.8	0.69	0.93	1.35	3.70	0.31	72.0
	1.0	0.67	0.88	0.21	3.67	0.32	67.3
	1.2	**0.81**	**0.98**	-0.81	3.72	**0.18**	58.8

Table 11. Hyperparameter sensitivity analysis for Beam Width (B), Top-K (K), and Nucleus (P).

Param	Value	Reward	Dist-2	Aux.
Beam Search (Aux: Avg Len)				
Beam (B)	2	1.90	0.94	79.4
	4	2.37	0.92	81.7
	8	**2.48**	**0.96**	81.4
	16	2.08	**0.96**	76.0
Top-K (Aux: Rep Rate)				
K Value	10	2.88	0.97	0.24
	30	-0.70	0.97	**0.17**
	50	**3.96**	**0.98**	0.20
	100	2.34	**0.98**	0.23
Top-P (Aux: Rep Rate)				
P Thresh	0.70	**6.63**	0.97	0.28
	0.85	4.34	**1.00**	0.24
	0.95	2.58	**1.00**	**0.12**
	0.99	3.77	0.96	0.20

A.2. Supplementary LLM ALignment

A.2.1. Perturbations for Reward Model Sensitivity Analysis

A.3. Supplementary Material - Sparse Autoencoders

Table 13. **The 'Universal Lobotomy' Ablation.** We evaluate the role of shared features by selectively ablating high-entropy (universal) vs. low-entropy (specific) units. Results show that high-entropy features are the primary drivers of cross-model translation.

Ablation Target	Feature Type	Cross-Model R^2	Perf. Drop
None (Baseline)	–	0.2233	–
Low Entropy	Model-Specific	0.1521	0.0712
High Entropy	**Universal**	**-0.1443**	**0.3675**

Table 12. Quantitative analysis of universality. **Top:** Alignment tax comparing USAE to independent SAEs. **Bottom:** Functional importance via ablations and resulting drop in cross-model reconstruction (R^2).

Experiment	**Metric**	**Value**	**Drop**
Universality Costs (Alignment Tax)			
Independent SAE	Reconstruction (R^2)	0.5820	—
Universal SAE (USAE)	Reconstruction (R^2)	0.4678	**0.1142 (11.4%)**
Feature Splitting	Avg. Jaccard Index	0.0193	—
Functional Ablation (Cross-Model R^2)			
Baseline	ResNet $\rightarrow$ ViT (R^2)	0.2233	—
Ablate Low-Entropy Features	Specific Feature Set	0.1521	$\downarrow$ 0.0712
Ablate High-Entropy Features	Universal Feature Set	-0.1443	$\downarrow$ **0.3675**

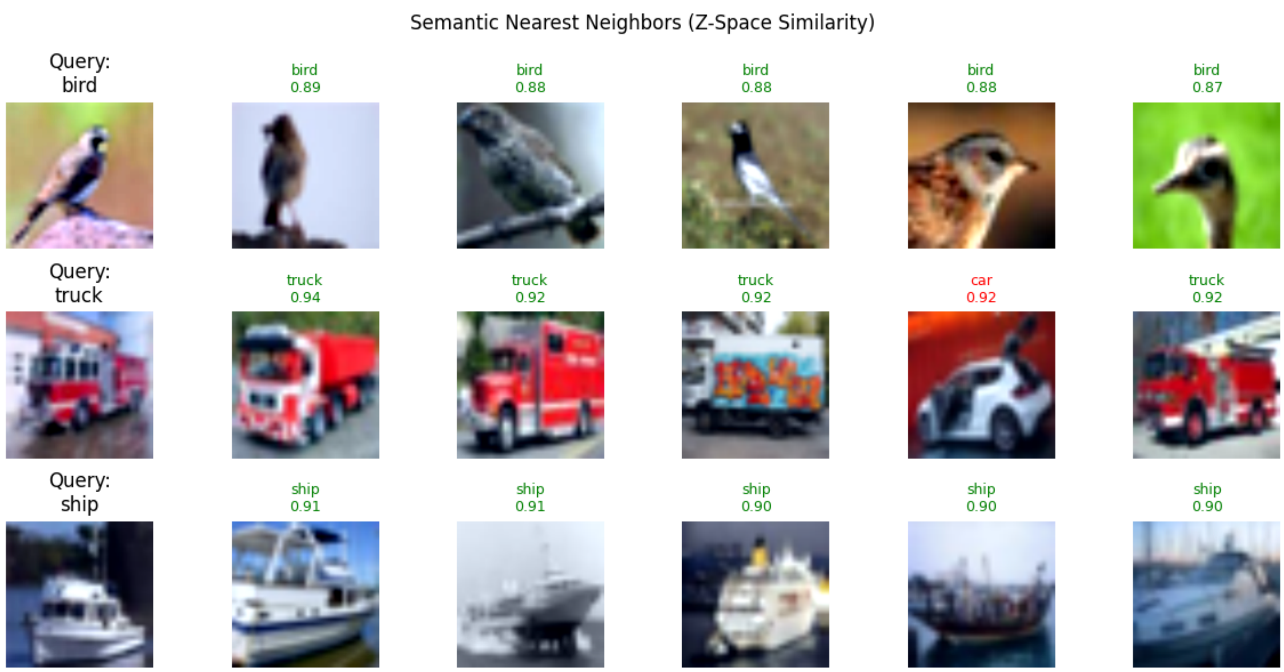

Figure 3. **Semantic coherence in the shared Z-space.** Nearest neighbor retrieval demonstrates that the Universal Sparse Autoencoder learns high-level semantic concepts aligned across architectures.

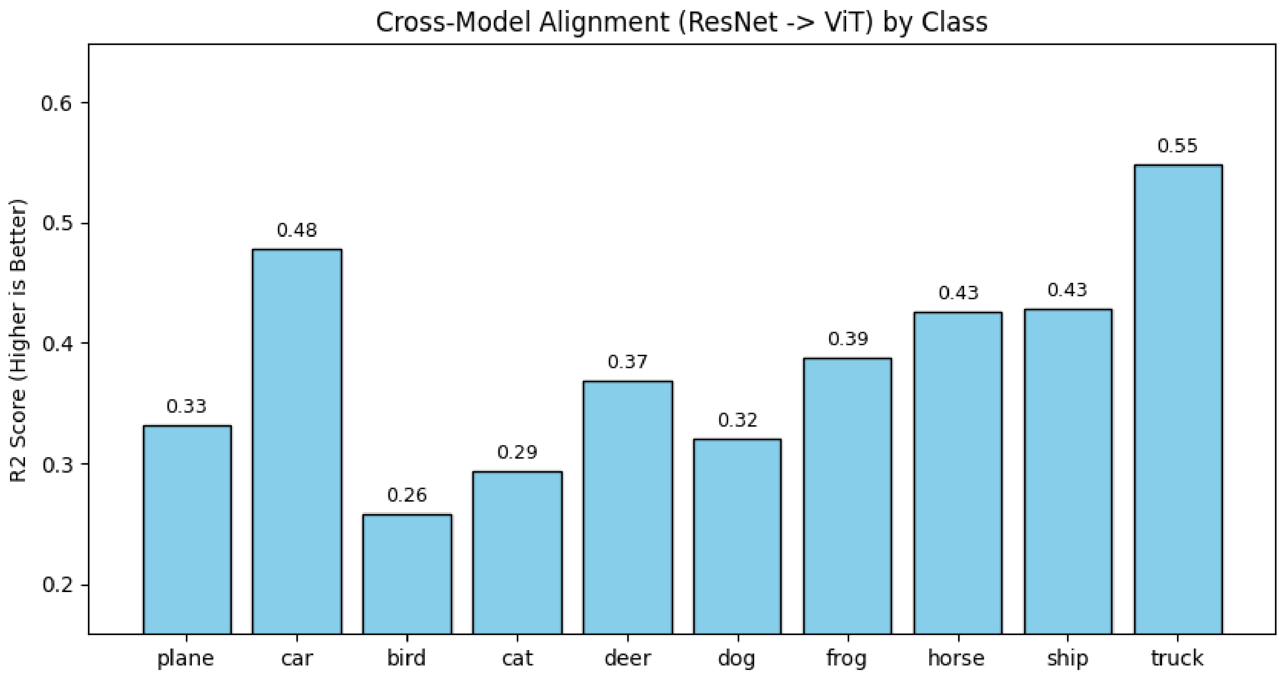

Figure 4. **Class-Stratified Alignment (ResNet $\rightarrow$ ViT).** Reconstruction quality (R^2) varies by category, with rigid objects (Trucks, Cars) showing significantly higher cross-model alignment than organic categories (Birds, Cats).

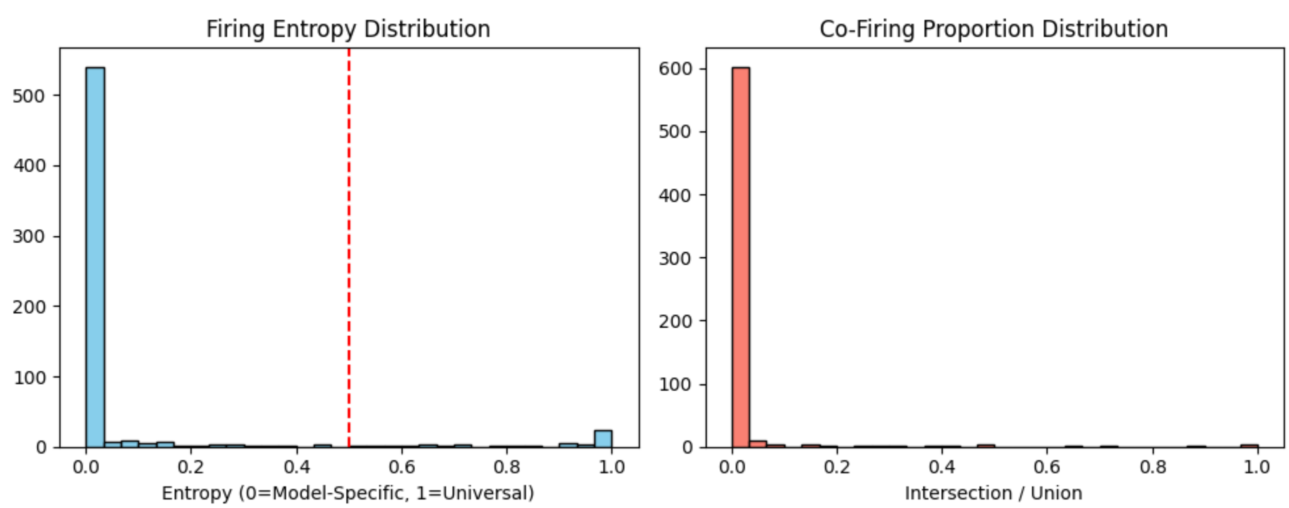

Figure 5. **Latent Space Statistics. Left:** The Normalized Firing Entropy distribution is bimodal, indicating a clear separation between universal (entropy $\approx$ 1) and specific features. **Right:** Co-firing proportions confirm distinct activation patterns.

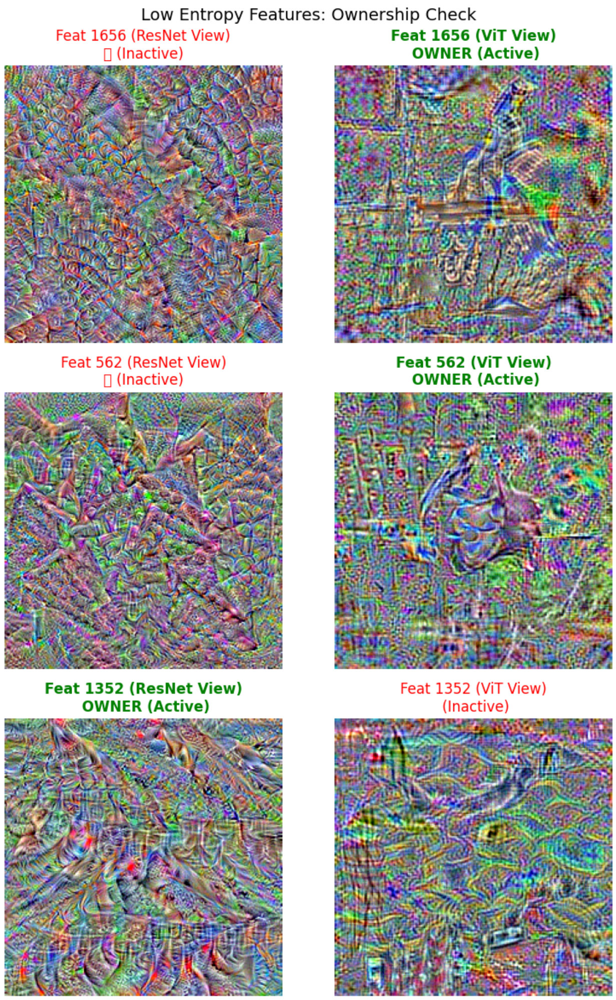

Figure 6. **Model-Specific Feature Visualization (Low Entropy).** These features ($FE \approx 0.0$) appear as "Inactive" noise in one model while remaining functionally active in the "Owner" model, confirming the separation of architecture-specific representations.